30-Day AI Engineer
Learning Roadmap

Personalized for a Python · SQL · ML · Visualization professional

2 hrs/day · Beginner-Intermediate · Video + Projects + Reading

30	60 hrs	4	1
Days	Total time	Projects	Live portfolio

■ Open Interactive Progress Tracker — mark daily tasks, view your completion %, get AI-powered action suggestions

---

ns

eddings, prompting

Day 1

The AI Engineer Landscape

› Watch: 'What is an AI Engineer?' (Swyx, 45 min)

› Read: AI Engineer roadmap on roadmap.sh

› Map your Python/SQL skills to AI use cases

Day 2

How LLMs Work

› Watch: Andrej Karpathy 'Intro to LLMs' (1 hr)

› Understand tokens, context window, temperature

› Play with tokenizer.openai.com

Day 3

Prompt Engineering

› Read: OpenAI Prompt Engineering Guide

› Practice zero-shot, few-shot, chain-of-thought

› Mini project: 10 prompts for data analysis tasks

Day 4

Python for AI

› async/await, Pydantic models, type hints

› Read: Real Python async guide (1 hr)

› Rewrite a past script using new patterns

Day 5

OpenAI & Anthropic APIs

› Set up API keys, call first endpoint

› Build a Python wrapper around chat completions

› Cost estimation: tokens → $$$

Day 6

Embeddings & Vector Search

› Watch: 'Embeddings Explained' (30 min)

› Generate embeddings for CSV data in Python

› Cosine similarity — hands-on notebook

Day 7

Week 1 Review + Mini Project

› Build: NL to SQL Assistant

› Input: natural language → Output: SQL query

› Push to GitHub with README

ouTube)

pace)

Suggested Action Plan

NL→SQL Assistant

You already know SQL deeply — let that be your unfair advantage. Start with a small SQLite database (use one from your past work or Kaggle). Connect OpenAI API → pass the schema as system context → let the LLM write queries. Test with 20 natural language questions. The gap between what you expect and what it generates IS the learning.

✓ Set up a venv + install openai, sqlite3
✓ Paste your DB schema into the system prompt
✓ Write 20 test questions across SELECT, GROUP BY, JOIN
✓ Log failures — they teach prompt tuning

PROJECT

NL to SQL Assistant

Leverage your SQL expertise. User types a question in English → LLM generates SQL → execute on sample DB → show results.

mory

with memory and tool use

Day 8

LangChain Introduction

› Watch: LangChain crash course (freeCodeCamp, 1 hr)

› Chains: LLMChain, SequentialChain

› Rebuild Day 5 API wrapper using LangChain

Day 9

Memory & Conversation History

› ConversationBufferMemory vs Summary memory

› Build a multi-turn chatbot in Python

› Read: LangChain memory docs (45 min)

Day 10

LangChain Tools & Toolkits

› Custom tool creation (wrapping your Pandas code)

› Built-in tools: Calculator, Search, Python REPL

› Mini: LLM that can run Python snippets

Day 11

Output Parsing & Structured Data

› Pydantic output parsers

› JSON mode — get structured responses

› Extract entities from text → DataFrame

Day 12

Vector Databases

› ChromaDB setup & basic CRUD

› Store and retrieve embeddings from a CSV

› Compare: Chroma vs Pinecone (read 30 min)

Day 13

Document Loaders & Indexing

› Load PDF, CSV, HTML with LangChain loaders

› Text splitting strategies (recursive, semantic)

› Index a company report and query it

Day 14

Week 2 Project Day

› Build: Data Analysis Chatbot

› Upload CSV → ask questions in plain English

› LLM interprets + runs Pandas → returns insights

deCamp

Suggested Action Plan

Data Analysis Chatbot

This project bridges your Power BI/Tableau skills with AI. Use a dataset you've analyzed before — you'll immediately know if the AI is giving sensible answers. LangChain's Pandas agent does the heavy lifting; your job is to write a good system prompt that constrains it to data analysis tasks only.

- ✓ Pick a real dataset you've worked with before
- ✓ Set up ChromaDB for column/schema retrieval
- ✓ Wrap Pandas agent with a Streamlit chat UI
- ✓ Test with at least 15 analytical questions

PROJECT

Data Analysis Chatbot

Upload any CSV → chat with your data. Uses LangChain + Pandas agent. Outputs summaries, statistics, and mini visualizations.

s

nd your first autonomous agent

Day 15

RAG Architecture Deep Dive

› Watch: RAG explained from scratch (30 min)

› Retrieval strategies: similarity, MMR, hybrid

› Diagram the full RAG pipeline on paper

Day 16

Build a RAG Pipeline

› Ingest docs → chunk → embed → store → retrieve

› Use ChromaDB + LangChain RetrievalQA

› Test with different chunk sizes (128, 512, 1024)

Day 17

RAG Evaluation

› RAGAS framework — faithfulness, relevancy scores

› Evaluate your Day 16 pipeline

› Read: 'Evaluating RAG' blog post (Trulens, 30 min)

Day 18

AI Agents & ReAct Pattern

› Watch: Agents explained — Andrew Ng (45 min)

› ReAct: Reason + Act loop

› Build a simple tool-using agent from scratch

Day 19

LangGraph Intro

› Nodes, edges, state — LangGraph basics

› Convert Day 18 agent to LangGraph

› Add conditional branching based on tool output

Day 20

Tool Use & Function Calling

› OpenAI function calling schema

› Build: agent with web search + Python execution

› Error handling and retry logic for agents

Day 21

Week 3 Project Day

› Build: AI Research Agent

› Input topic → searches web → retrieves PDFs → summarizes

› Uses RAG + agent loop + your ML knowledge

rning.ai)

lkthrough

Suggested Action Plan

AI Research Agent

The hardest part of this project is not the code — it's the evaluation. Your ML background gives you a huge edge here. Think of RAGAS scores like model metrics: faithfulness = precision, answer relevancy = recall. Tune chunk size and retrieval k as hyperparameters. Use a topic from your professional domain so you can judge output quality.

- ✓ Start with 3 documents max — scale up only after it works
- ✓ Use LangSmith to trace every agent step
- ✓ Set chunk_size=512, k=4 as baseline; grid-search from there
- ✓ Store RAGAS scores in a CSV and plot them — your viz skills shine here

---

PROJECT

AI Research Agent

Give it a topic. It searches the web, pulls documents, retrieves relevant sections, then synthesizes a structured report.

your portfolio

Day 22

FastAPI for AI Backends

› Build a REST API wrapping your RAG pipeline

› Streaming responses with Server-Sent Events

› Pydantic models for request/response validation

Day 23

Streamlit UI Layer

› Build a chat UI for your agent in Streamlit

› File upload + progress indicators

› Connect Streamlit to FastAPI backend

Day 24

Docker & Containerisation

› Dockerfile for your Python AI app

› docker-compose with app + vector DB

› Environment variables and secrets management

Day 25

Deploy to the Cloud

› Deploy to Render / Railway (free tier)

› Or HuggingFace Spaces for Streamlit apps

› Get your live URL — share with 3 people

Day 26

Observability & Logging

› LangSmith for LLM tracing and debugging

› Log latency, cost per query, error rates

› Add analytics dashboard (use your viz skills!)

Day 27–28

Final Capstone Build

› Build: Business Intelligence AI Assistant

› Write a README with architecture diagram

› Record a 3-min demo video

Day 29–30

Showcase & Next Steps

› Polish GitHub — pinned repos, profile README

› Write a LinkedIn post about what you built

› Create your 60-day learning plan (next phase)

Tim)

uide

Suggested Action Plan

Business Intelligence AI Assistant (Capstone)

This is your portfolio centrepiece — treat it like a product, not a homework assignment. Pick a domain you know (finance, sales, HR analytics) so your existing Power BI intuition tells you when the AI output is actually good. The goal: someone with zero technical background should be able to upload a spreadsheet and get boardroom-quality insights.

- ✓ Design the UI flow on paper before writing code
- ✓ Use your Power BI color palette — make it look professional
- ✓ Record a Loom walkthrough as a LinkedIn post
- ✓ Deploy on Day 25, iterate Days 26-28

PROJECT

BI AI Assistant (Capstone)

Upload business data (CSV/PDF) → RAG-powered Q&A; → auto-generated insights → embedded chart output. Deployed live with shareable URL.

What You Will Have After 30 Days

✓ Working RAG system built from scratch	✓ Deployed AI agent with a live public URL
✓ 4 portfolio projects on GitHub	✓ LLM + LangChain + LangGraph fluency
✓ Ability to evaluate AI system quality (RAGAS)	✓ FastAPI + Docker + cloud deployment skills
✓ AI-native Python patterns (async, Pydantic)	✓ Clear 60-day roadmap for continued growth

---

Prompt Comparison Analysis

1 Which roadmap feels more personalized?

This one — by design. Every project reuses existing skills: Day 7 builds an NL→SQL assistant because you already know SQL. Day 14 builds a data analysis chatbot because you know Pandas and visualization. The capstone is 'Power BI + AI' — so you'll have intuitions about quality before writing a line. A generic roadmap ignores all of this and starts from basics you mastered years ago.

2 Which roadmap would you actually follow?

This one. The 2 hr/day constraint is respected in every daily block. Projects produce something demonstrable by Day 7 — a live SQL assistant. Momentum from early wins is the biggest predictor of 30-day completion. Generic roadmaps fail because they front-load theory; here you ship working code in Week 1.

3 What role did context play in improving the result?

Context transformed the output from informational to actionable. Knowing your skills (Python, SQL, ML, viz) let us skip foundations entirely and start at the API level. Knowing your goal shaped Week 4 toward deployment, not research fine-tuning. Knowing the 2 hr/day limit made each block realistic. Without context, a roadmap is a syllabus. With context, it is a personal plan.

Interactive Progress Tracker

Track your daily progress, mark completed tasks, view your completion %, and get AI-powered action suggestions — all in one interactive dashboard.

Open your Progress Tracker →

Bookmark this link and check in every day.